TD1 — Terminal & Git fundamentals

Git, Python Environments & Coding Best Practices

MSA-DATB07-03 — 1 h 30 practical — **answer key**

Learning outcome. Install the tools, read and search files from the terminal, then create a Git repository, save changes with `git add` and `git commit`, read a history with `git log`, and undo mistakes with `git restore` and `git revert`.

Tip — how this TD works

Each exercise starts with **guided** steps: the command is given; *predict* what it will do, *run* it, *explain* the result. Then come **Your turn** questions: no command is given, adapt one from the exercise or from the slides.

Questions marked ★ are **key questions**: the kind of reasoning asked in the quiz. Write in `td1_answers.md` the command you typed and your answer, to revise later. This TD is not graded. The slides and the bash cheat-sheet are on https://github.com/msa-datb07-git: keep them open.

Part 0 — Install and set up *(20 min)*

Exercise 1 — Install Git, Python and VS Code

Follow the column of your system. If one of the tools is already installed, check its version and skip the step.

Windows

1. **VS Code**: https://code.visualstudio.com, default options.
2. **Git for Windows**: https://git-scm.com. Keep the defaults, except two screens: *default editor* → "Use Visual Studio Code"; *name of the initial branch* → "Override" and type `main`. Git for Windows installs Git *and* Git Bash, but **not** Python.
3. **Python 3.10 or newer**: https://www.python.org/downloads. On the first screen, tick **Add python.exe to PATH**.
4. In VS Code: Extensions (`Ctrl+Shift+X`) → install *Python* (Microsoft). Then `Ctrl+Shift+P` → *Terminal: Select Default Profile* → *Git Bash*.

macOS

1. Open *Terminal* (Spotlight: `Cmd+Space`, type "terminal"). Type `git --version`. If macOS offers to install the *command line developer tools*, accept: this installs Git.
2. **Python 3.10 or newer**: https://www.python.org/downloads, run the `.pkg` installer.
3. **VS Code**: https://code.visualstudio.com, drag it into *Applications*.
4. In VS Code: Extensions (`Cmd+Shift+X`) → install *Python* (Microsoft). Then `Cmd+Shift+P` → *Shell Command: Install 'code' command in PATH*.

Close VS Code completely, reopen it, and open a terminal with `Ctrl+‘` (the key left of 1). Check everything:

```
echo $0            # the name of the shell you are typing in (see below)
git --version      # git version 2.x
python --version   # Python 3.10 or newer   (macOS: python3 --version)
code --version     # the VS Code version
```

`echo` prints what you give it, and `$0` holds the name of the running program, here the shell. The answer must *contain* `bash` (Git Bash: `bash` in VS Code, `/usr/bin/bash` in its own window) or `zsh` (macOS, sometimes `-zsh`). An empty line means PowerShell; `$0` printed as is means cmd: change the terminal.

Pitfall — python opens the Microsoft Store, or says "not found"

Windows has a fake `python` that sends you to the Store. Fix: Windows *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*, turn off the two *python* entries, then reopen VS Code. If it still fails, run the Python installer again and tick *Add python.exe to PATH*.

Instructor note

Typical problems: `echo $0` prints nothing or an error → the terminal is PowerShell, the default profile was not changed (or VS Code was not restarted); `code: command not found` on macOS → the *Shell Command* step was skipped; `git --version` fine but `python` not found on Windows → *Add to PATH* was not ticked or the Store alias is active. On macOS, `python3` is the expected command at this stage.

Exercise 2 — Tell Git who you are

Once per computer, type these four commands with your own name and e-mail (the e-mail you will use for GitHub in session 3):

```
git config --global user.name  "Firstname Lastname"
git config --global user.email "firstname.lastname@student.example.edu"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

- `git config` changes a Git setting; `--global` means "for every repository on this computer".
- `user.name` and `user.email` are written on every commit you make, so that your teammates know who did what.
- `init.defaultBranch main`: every new repository starts with one line of work, called a *branch*. This gives it the name `main`. Older Git versions call it `master`; with this setting everybody has the same name.
- `core.editor "code --wait"`: sometimes Git needs you to write a text, for example a message. It then opens VS Code (`code`) and waits (`--wait`) until you save and close the tab before continuing.

Check with `git config --global --list`: you should see the four values.

Pitfall — a black screen full of ~ instead of VS Code

If Git opens this instead of VS Code, it is the *vim* editor (the last setting was not applied). To get out: press `Esc`, type `:wq` and `Enter` (save and quit). Then run the `core.editor` command above again.

Exercise 3 — Your course folder, and the TD material

1. Create a folder for this course inside *Documents*, and go into it:

   ```
   cd ~/Documents           # ~ = your home folder
   mkdir -p courses/git     # create courses, and git inside it (-p: the whole chain at
        once)
   cd courses/git
   pwd                      # check where you are
   ```

   If `cd ~/Documents` answers "No such file or directory" (common on Windows when Documents is in OneDrive), type `cd` followed by a space, drag your *Documents* folder from the file explorer onto the terminal, and press `Enter`. Then continue with `mkdir`.

2. From now on, a blue box **Where you should be** starts each exercise: run `pwd` and compare the end of the path (the beginning contains your own user name, it differs for everyone). If you are elsewhere, its `cd` command brings you back. If your *Documents* folder is in OneDrive, replace `~/Documents` by the path you used above.

3. Create your answers file *here*, in `courses/git`, next to the repositories you will download, not inside them: `code td1_answers.md`.
4. Download the TD material. `git clone <address>` downloads a copy of a project *with its whole history* and creates a folder with its name:

```
git clone https://github.com/msa-datb07-git/td1-shell-and-git.git
cd td1-shell-and-git
ls
```

Pitfall — CRYPT_E_NO_REVOCATION_CHECK on Windows

Some networks (company, campus) block a certificate check that Windows performs. If `git clone` fails with this message, run `git config --global http.schannelCheckRevoke false`, then clone again. The same problem with `curl` comes back in exercise 8, with its solution.

Instructor note

Expected: `pwd` shows `/c/Users/<name>/Documents/courses/git` (or `/c/Users/<name>/OneDrive/Documents/courses/git`) on Windows, `/Users/<name>/Documents/courses/git` on macOS; `ls` in the clone shows `README.md`, `project`, `shell-lab`.

Part A — Explore with the terminal *(30 min)*

Everything happens in the **terminal of VS Code** (Git Bash on Windows, zsh on macOS): no file explorer, no mouse. Start by going into the practice folder: `cd shell-lab`.

Exercise 4 — Look around

Where you should be: `pwd` ends with `courses/git/td1-shell-and-git/shell-lab`
If not: `cd ~/Documents/courses/git/td1-shell-and-git/shell-lab`

Guided. Predict, run, explain:

1. `pwd`
2. `ls`, then `ls -la`. Which names appear only with `-la`? What do they have in common?
3. `ls -la .secrets`, then `cat .secrets/note.txt`

Your turn ★ Go into `docs/archive/2019`, list what is there, and show the content of the file with `cat`, as you did for `note.txt`. Read it: what was the internal code name of the 2019 dashboard? Then come back to `shell-lab` with *one* `cd` command.

Answer

```
pwd                  # .../courses/git/td1-shell-and-git/shell-lab
ls ; ls -la          # only with -la: .env.example and .secrets (plus . and ..):
    names starting with a dot are hidden
ls -la .secrets ; cat .secrets/note.txt
cd docs/archive/2019 ; ls ; cat old_report.txt ; cd ../../..      # code name:
    PENGUIN
```

Check: `cd ../../..` (three levels up) or `cd -` / an absolute path are accepted; the code name is `PENGUIN` (line "Internal code name of the 2019 dashboard").

Exercise 5 — Read the data

Where you should be: `pwd` ends with `courses/git/td1-shell-and-git/shell-lab`
If not: `cd ~/Documents/courses/git/td1-shell-and-git/shell-lab`

Guided. Predict, run, explain:

1. `head -n 3 data/raw/sales_2024_q1.csv`. What are the columns?
2. `wc -l data/raw/sales_2024_q1.csv`. The answer is 41: how many *sales* does the file contain, and why not 41?

Your turn ★ How many sales are there over the *four* quarterly files together? One command, then explain your calculation.
Your turn Show the last 2 lines of the fourth-quarter file.

Answer

```
head -n 3 data/raw/sales_2024_q1.csv    # date,product,quantity,unit_price,region + 2 sales
wc -l data/raw/sales_2024_q1.csv        # 41 lines = 1 line of column names + 40 sales
wc -l data/raw/*.csv                    # 41, 56, 31, 66, total 194 -> 194 - 4 header lines = 190 sales
tail -n 2 data/raw/sales_2024_q4.csv
```

Check: 190, with the 4 header lines removed. Answering 194 shows the header was not taken into account.

Exercise 6 — Search

Where you should be: `pwd` ends with `courses/git/td1-shell-and-git/shell-lab`
If not: `cd ~/Documents/courses/git/td1-shell-and-git/shell-lab`

Guided. Predict, run, explain:

1. `grep -r PENGUIN .` — what does the `.` mean?
2. `grep -rl PENGUIN .` — what changes with `-l`?
3. `find .  -name "*.md"`

Your turn ★ Which file(s) mention an "api key", whatever the upper or lower case? Give only the file names.
Your turn List every CSV file of the folder with `find`.

Answer

```
grep -r PENGUIN .        # ./docs/archive/2019/old_report.txt:...   (. = from here)
grep -rl PENGUIN .       # only the file name
find . -name "*.md"      # glossary.md and the two meeting notes
grep -ril "api key" .    # ./docs/notes/meeting_2024-01-15.md   (-r every file, -i ignore case, -l names only)
find . -name "*.csv"     # the four files of data/raw
```

Check: the three options `-r`, `-i`, `-l` and the quotes around `"api key"` (two words). Without the quotes, `grep` looks for `api` in a file named `key`.

Exercise 7 — Send results to a file, chain commands

Where you should be: `pwd` ends with `courses/git/td1-shell-and-git/shell-lab`
If not: `cd ~/Documents/courses/git/td1-shell-and-git/shell-lab`

Guided. Predict, run, explain:

1. `grep -c ERROR logs/server.log`
2. `grep ERROR logs/server.log | head -n 3` — what does `|` do?
3. `grep ERROR logs/server.log > errors.txt`, then `wc -l errors.txt`
4. `grep WARN logs/server.log >> errors.txt`, then `wc -l errors.txt` again. Why is the number bigger? What would have happened with `>` instead of `>>`?

Your turn ★ How many `ERROR` lines does the *other* log file, `logs/app.log`, contain? Answer in two different ways: once with `-c`, once with a pipe `|`.
Your turn ★ Among the `ERROR` lines of `server.log`, how many are about a `timeout`? One line, with a pipe.

Answer

```
grep -c ERROR logs/server.log                    # 23
grep ERROR logs/server.log | head -n 3           # | gives the output of grep to head
grep ERROR logs/server.log > errors.txt ; wc -l errors.txt      # 23 errors.txt
grep WARN logs/server.log >> errors.txt ; wc -l errors.txt      # 61 errors.txt (23
    + 38)
grep -c ERROR logs/app.log                       # 4
grep ERROR logs/app.log | wc -l                  # 4
grep ERROR logs/server.log | grep -c timeout     # 6   (or ... | grep timeout | wc -l)
```

With `>` the file would have been emptied first: only the 38 WARN lines would remain.

Exercise 8 — Your computer and the network

Guided. Predict, run, explain. When a command differs between Windows and macOS, both are given.

1. `echo $HOME`, then `which python` (macOS: `which python3`)
2. `ping github.com` (Windows) or `ping -c 4 github.com` (macOS). Did all the test messages come back? What is the average time?
3. `curl -I https://github.com`: what is the status code, and what does it mean?
4. The current temperature in Paris:

```
curl -s "https://api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35&
    current=temperature_2m" | python -m json.tool
```

Find the temperature in the answer. Why are the quotes around the address needed?

Pitfall — curl: (35) ... CRYPT_E_NO_REVOCATION_CHECK

On Windows, on some networks (companies, some campuses), `curl` stops with this error. The site is fine: the network inspects secure connections, and Windows then cannot check whether the site's certificate was cancelled ("revoked"), so it refuses to connect. Add the option that skips this check:

```
curl --ssl-no-revoke -s "https://api.open-meteo.com/v1/forecast?latitude=48.85&
    longitude=2.35&current=temperature_2m"
```

To never type it again, write the option once in curl's configuration file (`>>` adds a line at the end), and check it:

```
echo "ssl-no-revoke" >> ~/.curlrc
cat ~/.curlrc                          # must print: ssl-no-revoke
```

Python says "Expecting value: line 1 column 1"? `-s` (silent) also hides curl's error messages: Python then receives an empty answer. Run the `curl` command again *without* `-s` to see the real error.

Your turn ★ Get the current temperature of another city of your choice: find its latitude and longitude (a search engine gives them, for example London: 51.51 and -0.13) and change the address. Give the command and the temperature.

Answer

```
echo $HOME ; which python          # /c/Users/<name> ; the python found first in
    PATH
ping github.com                    # 4 replies, 0% loss, ~15-40 ms   (macOS: ping -
    c 4 github.com)
curl -I https://github.com         # HTTP/2 200 (or HTTP/1.1 200 OK): the page
    exists and answers normally
curl -s "https://api.open-meteo.com/v1/forecast?latitude=51.51&longitude=-0.13&
    current=temperature_2m" | python -m json.tool
```

Check for the last question: `latitude` and `longitude` changed in the address, the quotes kept, the value of `temperature_2m` reported. Campus networks often block `ping`: 100 % loss while the browser works means the network blocks the test, not that GitHub is down.

Part B — Your first repository *(35 min)*

Exercise 9 — Your project, commit by commit

Go back to your course folder: `cd ../..` (you are now in `courses/git`). You build your own project from scratch.

Where you should be: `pwd` ends with `courses/git`
If not: `cd ~/Documents/courses/git`

Guided: the first two commits. Run each command, and explain what it did.

```
mkdir my_sales
cd my_sales
git init                                    # turn this folder into a Git repository
git status                                  # what does Git say?
code README.md                              # write one line: # My sales analysis, save
git status                                  # what changed?
git add README.md
git status                                  # and now?
git commit -m "Add project README"
git log --oneline
mkdir data
cp ../td1-shell-and-git/project/data/sales.csv data/
git add data
git commit -m "Add Q1 2024 sales data"
git log --oneline
```

1. What does `git status` say about `README.md` before `git add`, after `git add`, and after `git commit`?

Your turn: the next commits, on the model of the first two, each with a clear message.

2. Create `analysis.py` with this function, then commit it:

```
import csv


def load_sales(path):
    """Read a sales CSV file and return a list of dictionaries (one per row)."""
    with open(path, newline="") as f:
        return list(csv.DictReader(f))
```

3. ★ Add this second function at the end of `analysis.py` and save. *Before committing*, run `git diff`, then `git add analysis.py`, then `git diff` again, then `git diff --staged`. What does each of the three show? Then commit.

```
def total_revenue(rows):
    """Sum quantity * unit_price over all rows."""
    return sum(float(r["quantity"]) * float(r["unit_price"]) for r in rows)
```

4. Create a file `.gitignore` containing these three lines, then commit it:

```
__pycache__/
outputs/
.env
```

5. ★ Create a file `.env` containing `API_KEY=secret`, and run `git status`. Is `.env` listed? Why?

6. ★ Write yourself a third function `count_sales(rows)` that returns the number of sales (the number of rows), and commit it with a clear message.

Answer

```
# guided part: README.md is "untracked" -> "Changes to be committed: new file" -> "
    nothing to commit, working tree clean"
git add analysis.py ; git commit -m "Add load_sales function"
git diff            # the new lines, not staged yet
git add analysis.py
git diff            # now empty: nothing left that is not staged
git diff --staged   # the new lines again: what the next commit will contain
git commit -m "Add total_revenue function"
git add .gitignore ; git commit -m "Add .gitignore"
git status          # .env is not listed: it matches a line of .gitignore
git add analysis.py ; git commit -m "Add count_sales function"
```

```
def count_sales(rows):
    """Number of sales (one row = one sale)."""
    return len(rows)
```

Expected in `td1_my_sales_log.txt`: 6 commits, each with a message that says what it does.

Exercise 10 — Read someone else's history

Go to the downloaded repository: `cd ../td1-shell-and-git`.

Where you should be: `pwd` ends with `courses/git/td1-shell-and-git`
If not: `cd ~/Documents/courses/git/td1-shell-and-git`

Guided. Predict, run, explain:

1. `git log --oneline`: how many commits?
2. `git log --stat`: one commit added a file that should never have been committed. Which commit, which file? (A long list waits at the bottom of the screen: `Space` for the next page, `q` to quit.)
3. `git show da319b1`: what is in that file? Why is it a problem?

Your turn ★ Which commit added the function `monthly_revenue`? Who wrote it, and when? Show only that commit, with its changes.

Answer

```
git log --oneline       # 10 commits
git log --stat          # da319b1 "Add config for the reporting API" adds project/
    config.ini
git show da319b1        # api_key = sk-live-... : a secret, now in the history
git log --oneline       # 79ee015 Add monthly_revenue function
git show 79ee015        # Alice Martin, Mon Sep 1 17:00 2025, + def monthly_revenue(
    rows): ...
```

Check: identifier `79ee015`, author Alice Martin, date 1 September 2025, and `git show 79ee015` (or `git log -n 1 -p 79ee015`).

Exercise 11 — Undo

Where you should be: `pwd` ends with `courses/git/td1-shell-and-git`
If not: `cd ~/Documents/courses/git/td1-shell-and-git`

Still in `td1-shell-and-git`. If `git status` lists files you created in Part A (`shell-lab/errors.txt`), ignore them: they do not affect what follows.

1. **Guided — staged too early.** Add a line to `README.md` and save. Then predict, run, explain:

   ```
   git add README.md
   git status
   git restore --staged README.md     # is the line still in the file?
   git status
   git restore README.md              # and now?
   ```

2. **Guided — a bad commit: revert it.** Predict, run, explain:

   ```
   git revert da319b1     # VS Code opens with a message: keep it, save, close the tab
   git log --oneline
   ls project
   ```

   Is `config.ini` still in the folder? Is it still in the history (`git show da319b1`)? Did the later commits (for example `monthly_revenue`) disappear?

3. **Your turn ★ — the exception.** Add a line to `README.md` and commit it with the message "oops". Using the slides, undo this last commit *while keeping your change*, check with `git status` where the change is, then commit it again with a proper message.

4. ★ In one or two sentences: why is `git revert` the default way to undo a commit, and `git reset` only an exception?

Answer

1. After `restore --staged`: the line is still in the file, just no longer staged. After `git restore README.md`: the line is gone for good (it was never committed).
2. A new commit `Revert "Add config for the reporting API"` appears on top; `config.ini` is no longer in `project/`, but `git show da319b1` still shows it: the history is kept. The later commits are all still there: revert only cancels the changes of da319b1. In real life the key must also be revoked on the service.
3. `git add README.md ; git commit -m "oops" ; git reset --soft HEAD~1 ; git status` (the change is back in "Changes to be committed") `; git commit -m "Add a line to the README"`. Check: `--soft` (a `--hard` would lose the change), `HEAD~1`, no "oops" left in `td1_undo_log.txt`.
4. `revert` adds a commit and erases nothing, so nothing can be lost and teammates who already have the history are not affected; `reset` removes commits, acceptable only for a commit that never left your laptop.

Part C — Checklist *(5 min)*

1. What does `Ctrl+C` do in a terminal? How do you copy text instead?
2. What is the difference between `>` and `>>`?
3. Why do we undo a commit with `git revert` by default, and not with `git reset`?

Answer

1. It stops the running command. Copy with `Ctrl+Insert` (Windows) or `Cmd+C` (macOS), paste with `Shift+Insert` or `Cmd+V`.
2. `>` empties the file then writes into it; `>>` adds at the end and keeps the old content.
3. `revert` adds a commit and erases nothing; `reset` removes commits, acceptable only for a commit that never left your laptop.

Deliverable

Nothing to send: TD1 is not graded. Keep your `td1_answers.md`: the ★ questions are the kind of reasoning asked in the quiz. Keep your folders as they are.